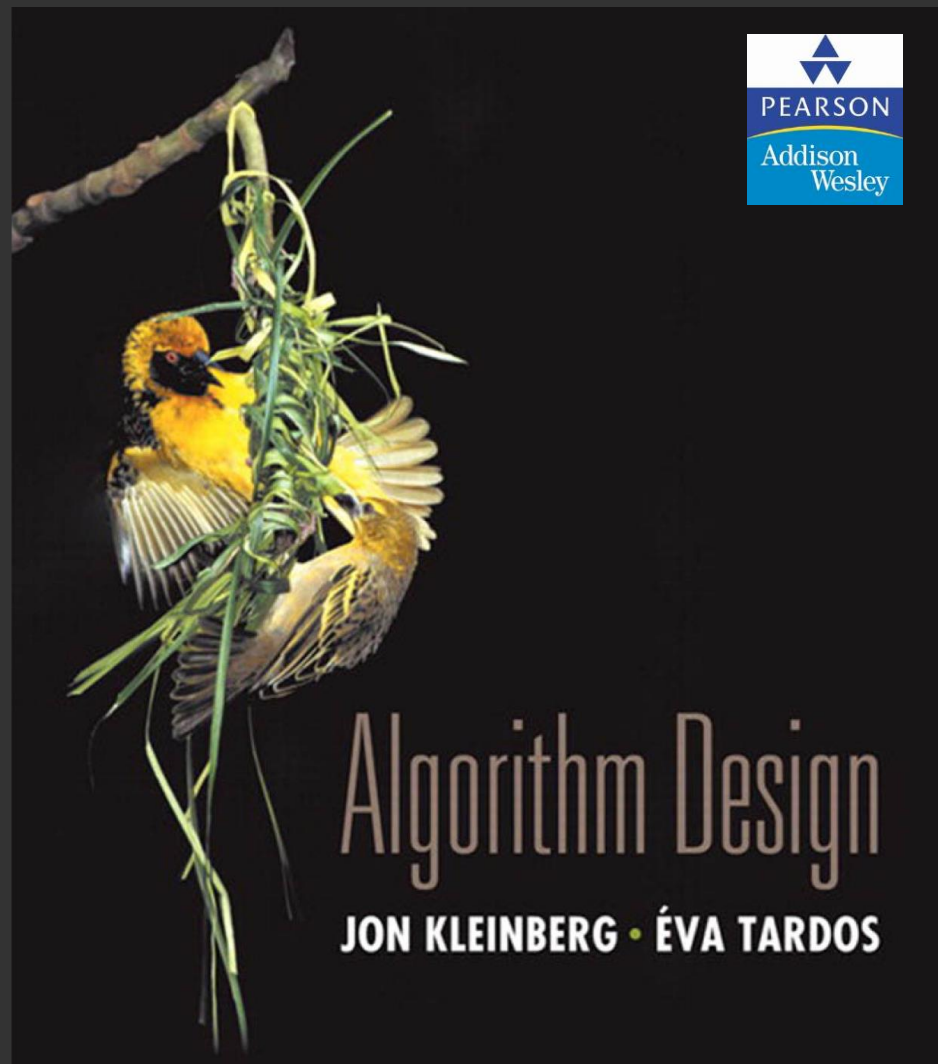




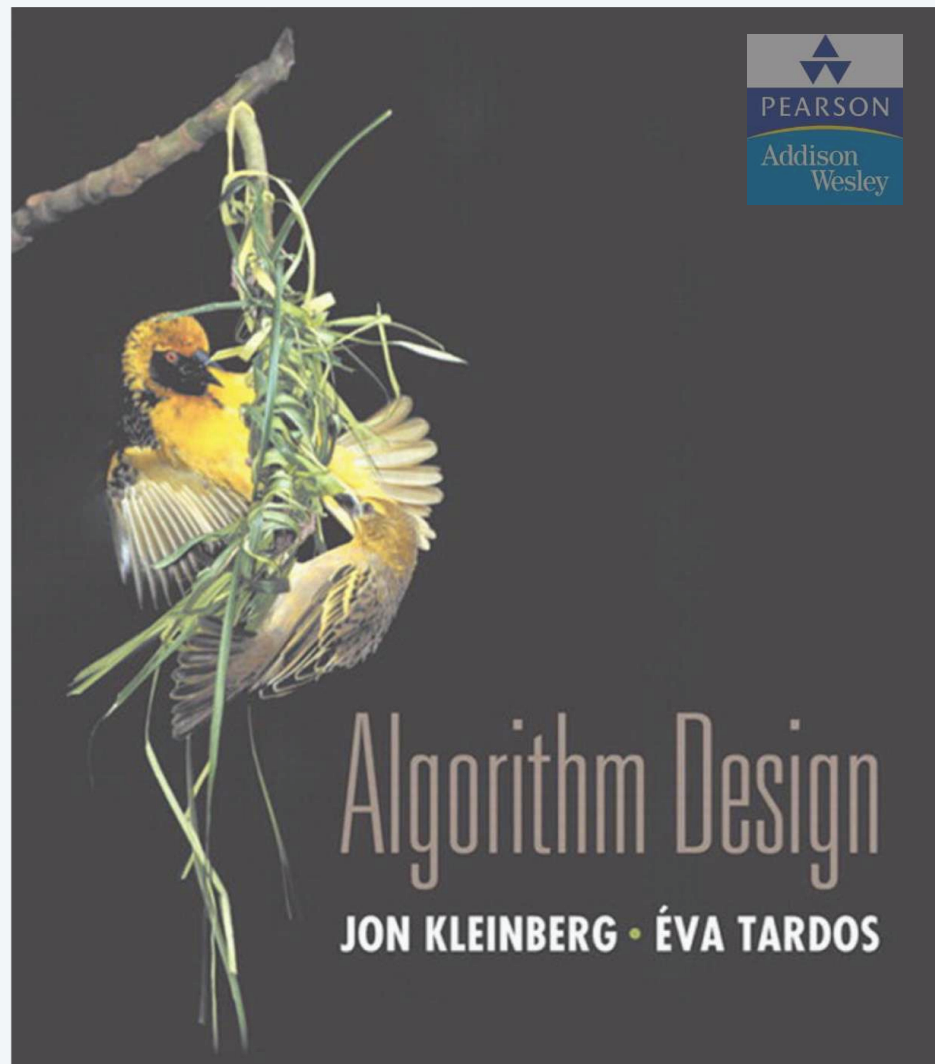
8. INTRACTABILITY I

- *poly-time reductions*

Lecture slides by Kevin Wayne

Copyright © 2005 Pearson-Addison Wesley

<http://www.cs.princeton.edu/~wayne/kleinberg-tardos>



SECTION 8.1

8. INTRACTABILITY I

- *poly-time reductions*

Algorithm design patterns and antipatterns

Algorithm design patterns.

- Greedy.
- Divide and conquer.
- Dynamic programming.
- Duality.
- **Reductions.**
- Local search.
- Randomization.

Algorithm design antipatterns. *è abbastanza improbabile che si possa risolvere così*

- **NP-completeness.** *$O(n^k)$ algorithm unlikely.*
- **PSPACE-completeness.** *$O(n^k)$ certification algorithm unlikely.*
- Undecidability. *No algorithm possible.*

✗

Classify problems according to computational requirements

Q. Which problems will we be able to solve in practice?

un problema si può risolvere

A working definition. Those with poly-time algorithms. *velocemente se esiste un algoritmo polinomiale che risolve l'istanza*



von Neumann
(1953)



Nash
(1955)



Gödel
(1956)



Cobham
(1964)



Edmonds
(1965)



Antonio Chisso
(1966)

Turing machine, word RAM, uniform circuits, ...

Theory. Definition is broad and robust.

constants tend to be small, e.g., $3n^2$

Practice. Poly-time algorithms scale to huge problems.

Quali sono i problemi che si possono risolvere

in tempo polinomiale??

Classify problems according to computational requirements

Q. Which problems will we be able to solve in practice?

A working definition. Those with poly-time algorithms.

yes	probably no
shortest path	longest path
min cut	max cut
2-satisfiability	3-satisfiability
planar 4-colorability	planar 3-colorability
bipartite vertex cover	vertex cover
matching	3d-matching
primality testing	factoring
linear programming	integer linear programming

Classify problems

Desiderata. Classify problems according to those that can be solved in polynomial time and those that cannot.

Provably requires exponential time.

- Given a constant-size program, does it halt in at most k steps?
- Given a board position in an n -by- n generalization of checkers, can black guarantee a win?

input size = $c + \log k$

using forced capture rule



Alan designed the perfect computer



Frustrating news. Huge number of fundamental problems have defied classification for decades.

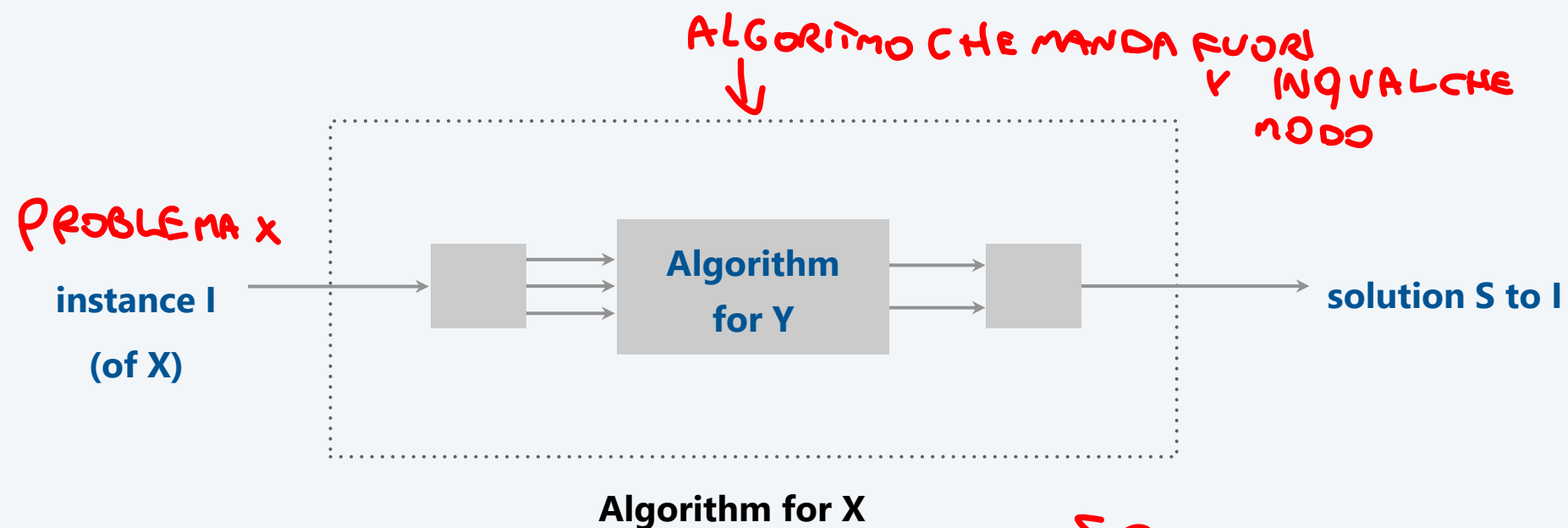
Poly-time reductions

Desiderata'. Suppose we could solve problem Y in polynomial time. What else could we solve in polynomial time?

Reduction. Problem X **polynomial-time (Cook) reduces to** problem Y if arbitrary instances of problem X can be solved using:

- Polynomial number of standard computational steps, plus
- Polynomial number of calls to oracle that solves problem Y .

computational model supplemented by special piece of hardware that solves instances of Y in a single step



SO RISOLVERE X SE SO RISOLVERE
UNA GENERICA ISTANZA Y

Poly-time reductions

Desiderata'. Suppose we could solve problem Y in polynomial time. What else could we solve in polynomial time?

Reduction. Problem X **polynomial-time (Cook) reduces to** problem Y if arbitrary instances of problem X can be solved using:

- Polynomial number of standard computational steps, plus
- Polynomial number of calls to oracle that solves problem Y .

Y + DIFFICILE DI X

Notation. $X \leq_p Y$.

Note. We pay for time to write down instances of Y sent to oracle $\Rightarrow$ instances of Y must be of polynomial size.

Novice mistake. Confusing $X \leq_p Y$ with $Y \leq_p X$.

Poly-time reductions

Design algorithms. ^{SE X RIDUCE Y} If $X \leq_p Y$ and Y can be solved in polynomial time, then X can be solved in polynomial time.

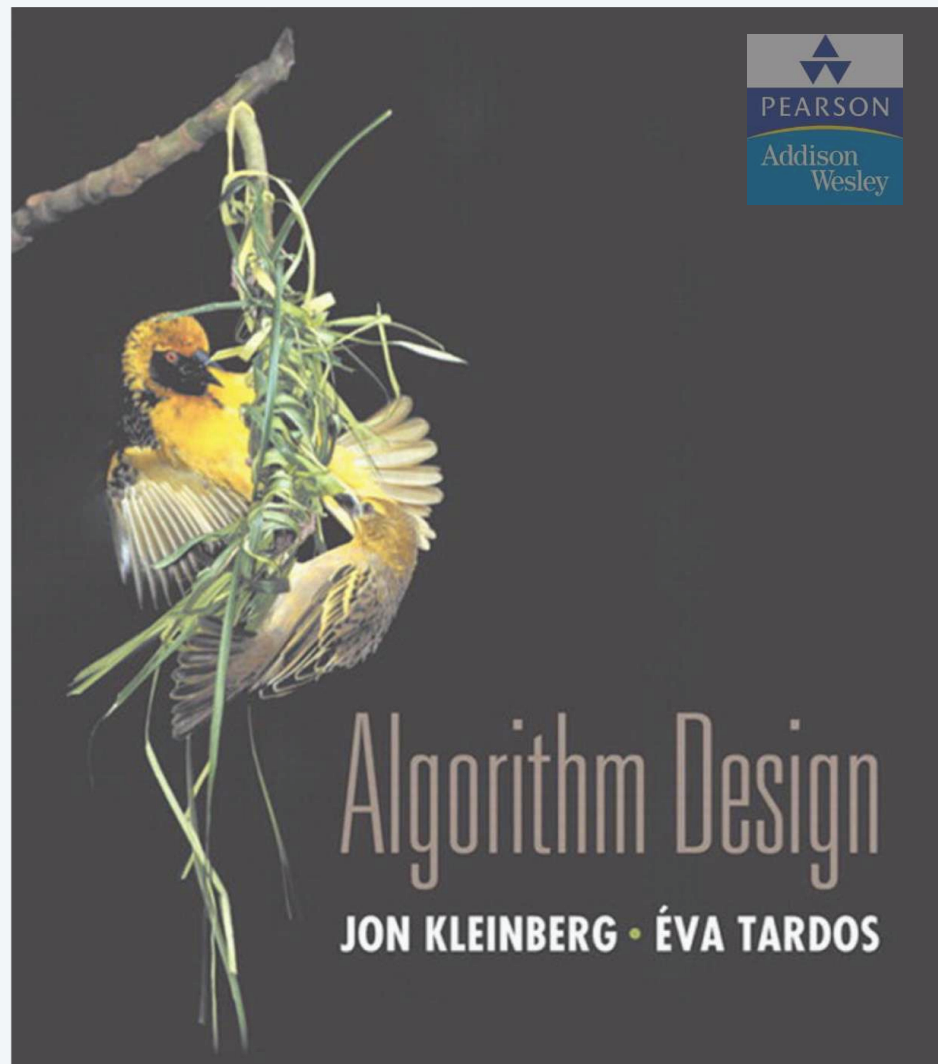
Diciamo che un problema X si riduce a un problema Y (scriviamo $X \leq_p Y$) se posso trasformare ogni istanza di X in un'istanza di Y in tempo polinomiale, in modo che risolvere l'istanza di Y mi dia anche la soluzione dell'istanza di X .

Establish intractability. If $X \leq_p Y$ and X cannot be solved in polynomial time, then Y cannot be solved in polynomial time.

Establish equivalence. If both $X \leq_p Y$ and $Y \leq_p X$, we use notation $X \equiv_p Y$. In this case, X can be solved in polynomial time iff Y can be.

CATALOGARE PROBLEMI RISPETTO ALLA DIFFICOLTÀ

Bottom line. Reductions classify problems according to **relative** difficulty.

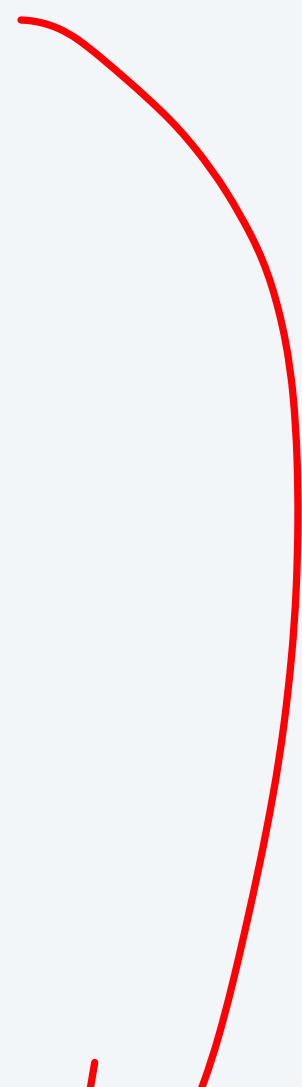


SECTION 8.1

8. INTRACTABILITY I

- *packing and covering problems*

ESEMPI CON PROBLEMI
REALI



Independent set



INDEPENDENT-SET. Given a graph $G = (V, E)$ and an integer k , is there a subset of k (or more) vertices such that no two are adjacent?

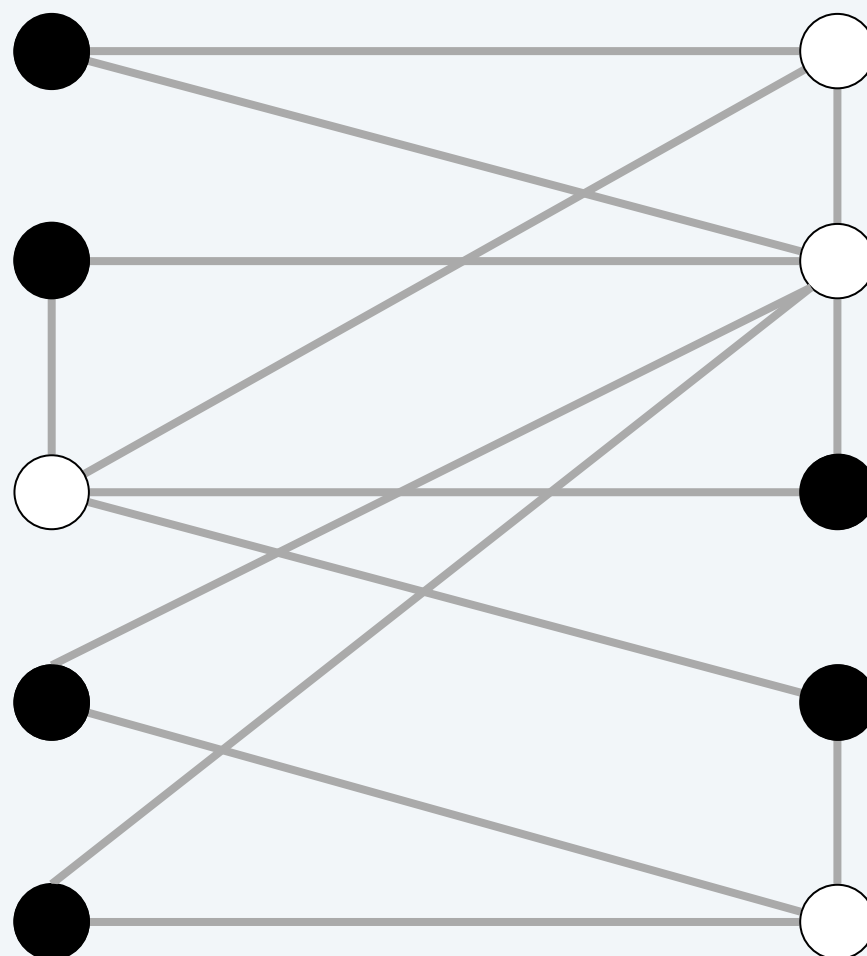
Ex. Is there an independent set of size ≥ 6 ?

DATO $k \exists$ UN INDIENE INDIP.

Ex. Is there an independent set of size ≥ 7 ?

E.C. SIA $\geq k$?

INSIEMI VERTICI E.C. SONO INDIPENDENTI



independent set of size 6

Vertex cover

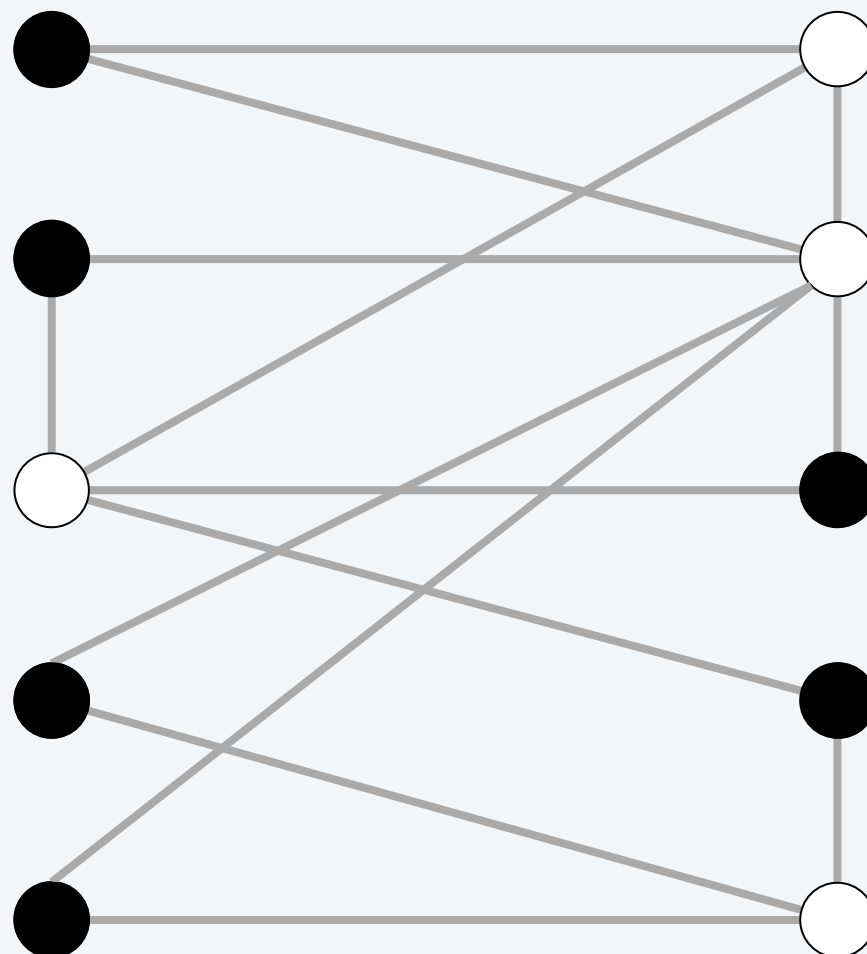
VERTEX-COVER. Given a graph $G = (V, E)$ and an integer k , is there a subset of k (or fewer) vertices such that each edge is incident to at least one vertex in the subset?

Ex. Is there a vertex cover of size ≤ 4 ?

Ex. Is there a vertex cover of size ≤ 3 ?



NON DEVI LASCIARE
ARCHI SCOPERTI

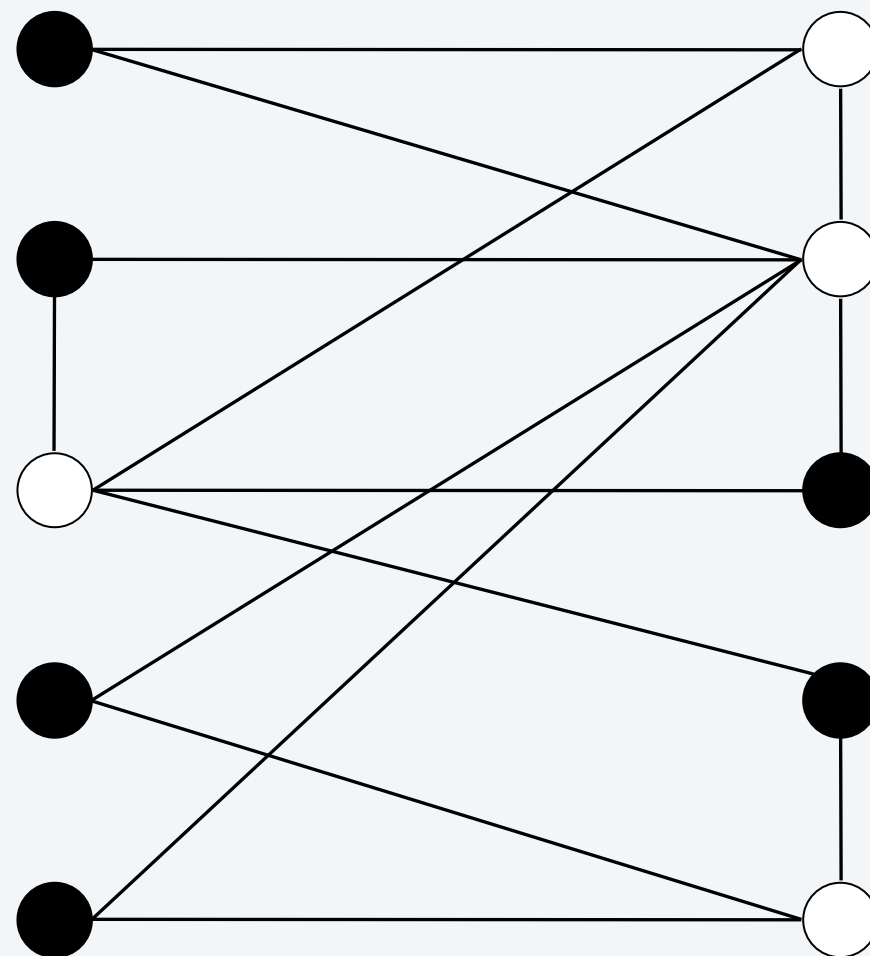


● independent set of size 6
○ vertex cover of size 4

Vertex cover and independent set reduce to one another

Theorem. $\text{INDEPENDENT-SET} \equiv_p \text{VERTEX-COVER}$. ← DAL PUNTO DI VISTA POLINOM.

Pf. We show S is an independent set of size k iff $V - S$ is a vertex cover of size $n - k$. IDUE SI RIDUCONO



● independent set of size 6
○ vertex cover of size 4

Vertex cover and independent set reduce to one another

Theorem. $\text{INDEPENDENT-SET} \equiv_p \text{VERTEX-COVER}$.

Pf. We show S is an independent set of size k iff $V - S$ is a vertex cover of size $n - k$.

$\Rightarrow$

- Let S be any independent set of size k .
- $V - S$ is of size $n - k$.
- Consider an arbitrary edge $(u, v) \in E$.
- S independent $\Rightarrow$ either $u \notin S$, or $v \notin S$, or both.
 $\Rightarrow$ either $u \in V - S$, or $v \in V - S$, or both.
- Thus, $V - S$ covers (u, v) . ▪

Vertex cover and independent set reduce to one another

Theorem. $\text{INDEPENDENT-SET} \equiv_p \text{VERTEX-COVER}$.

Pf. We show S is an independent set of size k iff $V - S$ is a vertex cover of size $n - k$.

ANCHE AL CONTRARIO

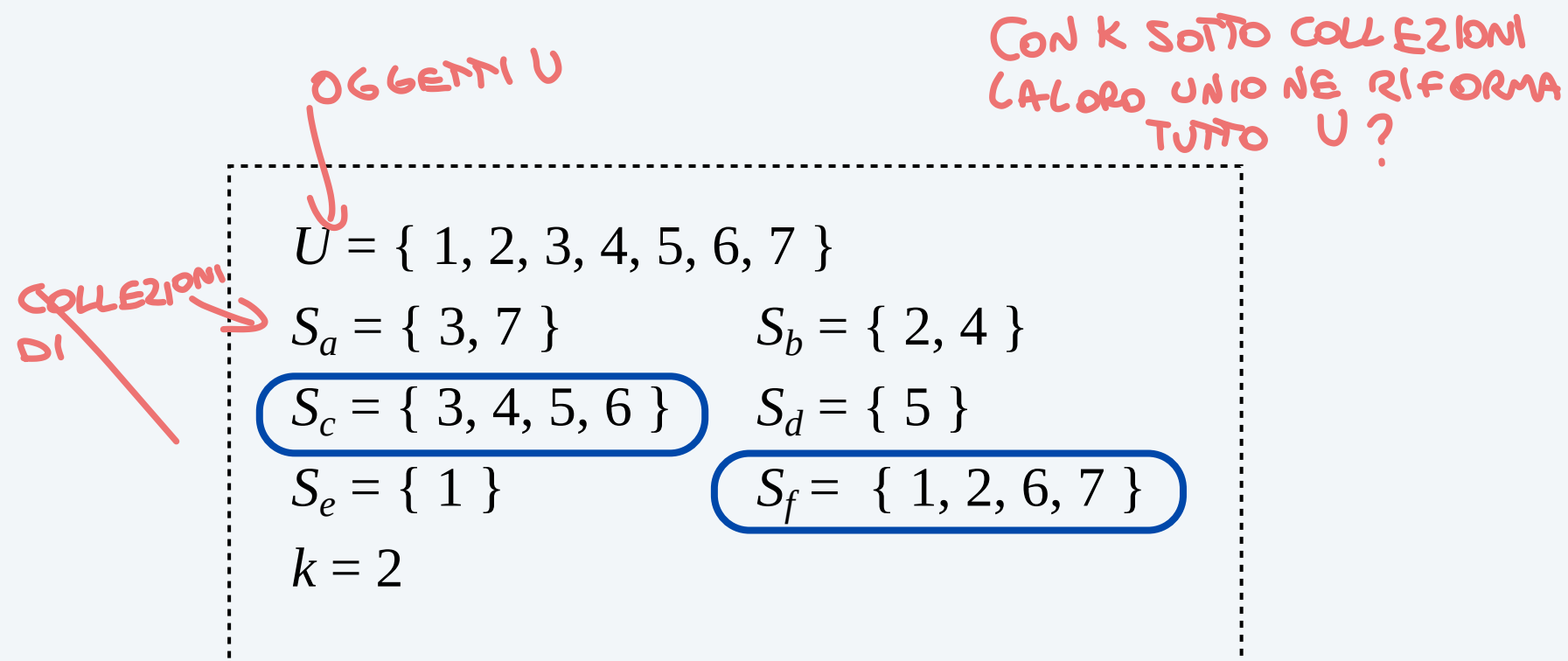

- Let $V - S$ be any vertex cover of size $n - k$.
- S is of size k .
- Consider an arbitrary edge $(u, v) \in E$.
- $V - S$ is a vertex cover $\Rightarrow$ either $u \in V - S$, or $v \in V - S$, or both.
 $\Rightarrow$ either $u \notin S$, or $v \notin S$, or both.
- Thus, S is an independent set. ▪

Set cover

SET-COVER. Given a set U of elements, a collection S of subsets of U , and an integer k , are there $\leq k$ of these subsets whose union is equal to U ?

Sample application.

- m available pieces of software.
- Set U of n capabilities that we would like our system to have.
- The i^{th} piece of software provides the set $S_i \subseteq U$ of capabilities.
- Goal: achieve all n capabilities using fewest pieces of software.



a set cover instance

One more example

Given the universe $U = \{ 1, 2, 3, 4, 5, 6, 7 \}$ and the following sets, which is the minimum size of a set cover?

$$U = \{ 1, 2, 3, 4, 5, 6, 7 \}$$

$$S_a = \{ 1, 4, 6 \}$$

$$S_b = \{ 1, 6, 7 \}$$

$$S_c = \{ 1, 2, 3, 6 \}$$

$$S_d = \{ 1, 3, 5, 7 \}$$

$$S_e = \{ 2, 6, 7 \}$$

$$S_f = \{ 3, 4, 5 \}$$

minimum size: 3

GENERALIZA
VERTEX-COVER

Vertex cover reduces to set cover

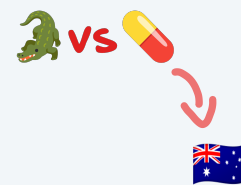


Theorem. $\text{VERTEX-COVER} \leq_p \text{SET-COVER}$.

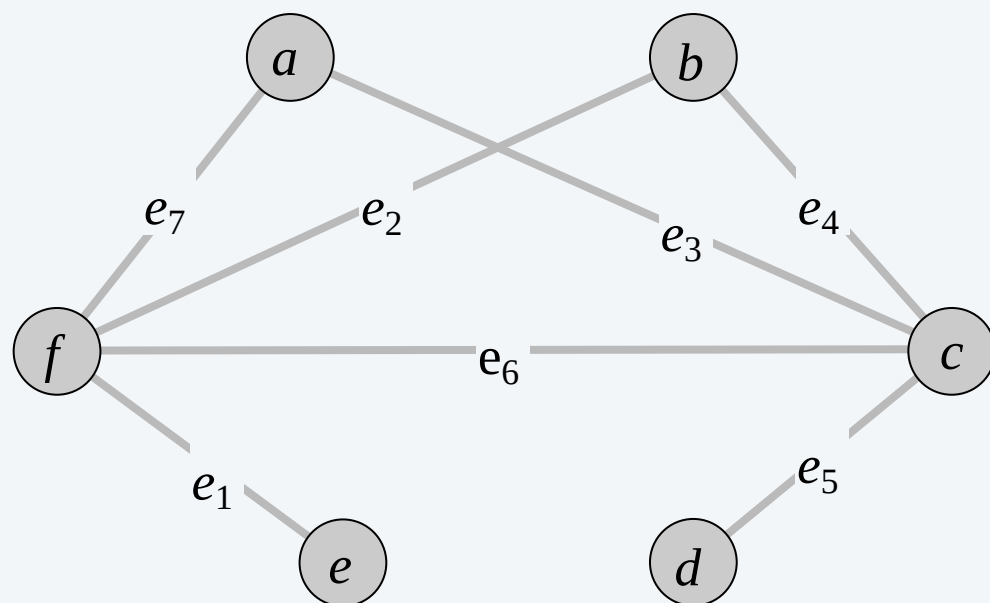
Pf. Given a VERTEX-COVER instance $G = (V, E)$ and k , we construct a SET-COVER instance (U, S, k) that has a set cover of size k iff G has a vertex cover of size k .

Construction.

- Universe $U = E$.
- Include one subset for each node $v \in V$: $S_v = \{e \in E : e \text{ incident to } v\}$.



gli archi rappresentano gli elementi dell'insieme



vertex cover instance
($k = 2$)

$U = \{ 1, 2, 3, 4, 5, 6, 7 \}$	
$S_a = \{ 3, 7 \}$	$S_b = \{ 2, 4 \}$
$S_c = \{ 3, 4, 5, 6 \}$	$S_d = \{ 5 \}$
$S_e = \{ 1 \}$	$S_f = \{ 1, 2, 6, 7 \}$

set cover instance
($k = 2$)

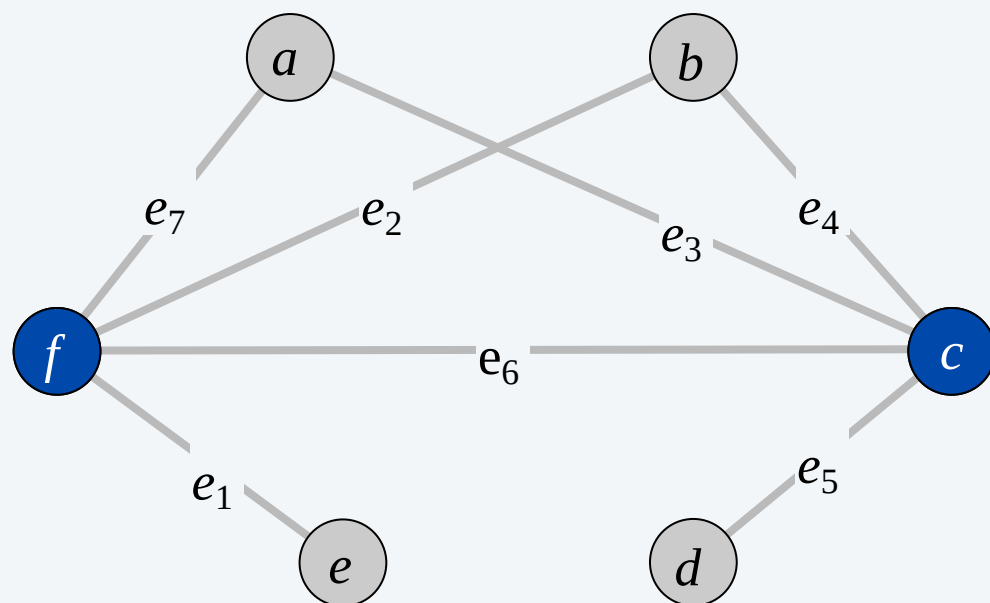
Vertex cover reduces to set cover

Lemma. $G = (V, E)$ contains a vertex cover of size k iff (U, S, k) contains a set cover of size k .

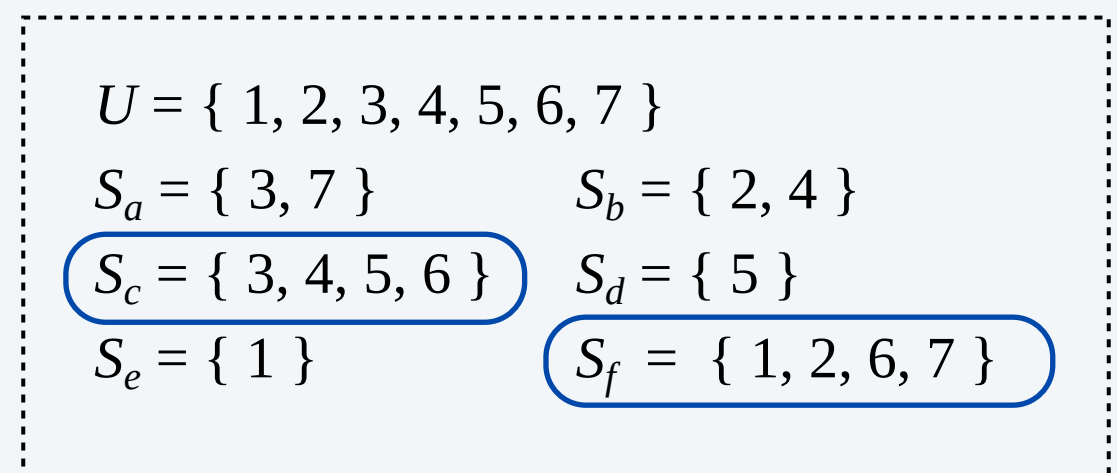
Pf. $\Rightarrow$ Let $X \subseteq V$ be a vertex cover of size k in G .

- Then $Y = \{ S_v : v \in X \}$ is a set cover of size k . ▪

“yes” instances of VERTEX-COVER
are solved correctly



vertex cover instance
($k = 2$)



set cover instance
($k = 2$)

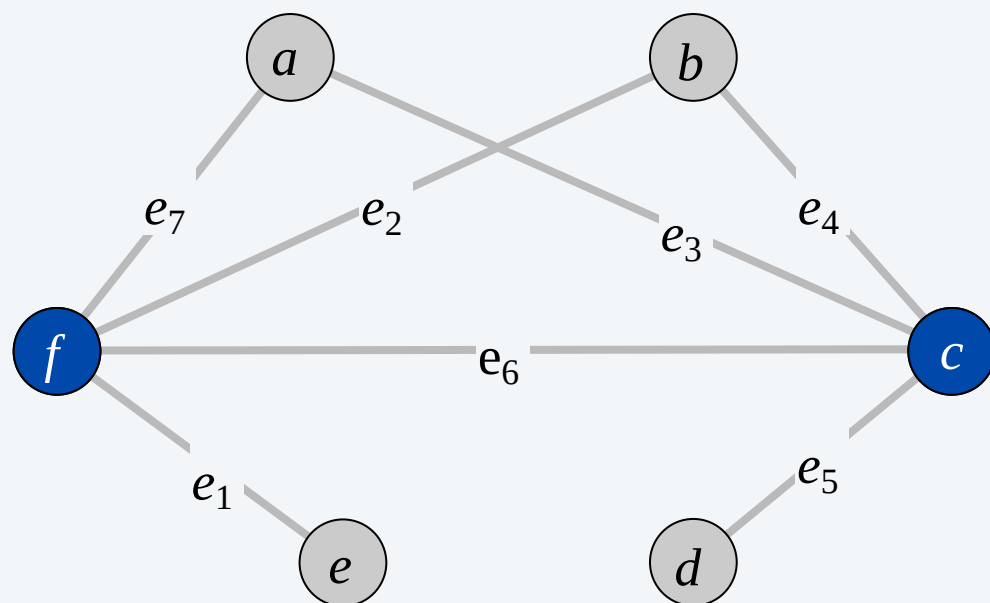
Vertex cover reduces to set cover

Lemma. $G = (V, E)$ contains a vertex cover of size k iff (U, S, k) contains a set cover of size k .

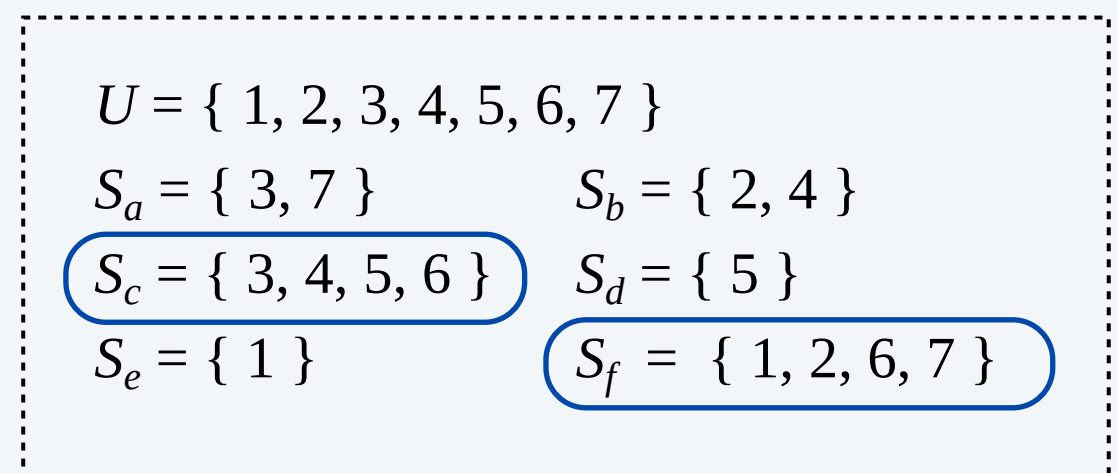
Pf. $\Leftarrow$ Let $Y \subseteq S$ be a set cover of size k in (U, S, k) .

- Then $X = \{ v : S_v \in Y \}$ is a vertex cover of size k in G . ▪

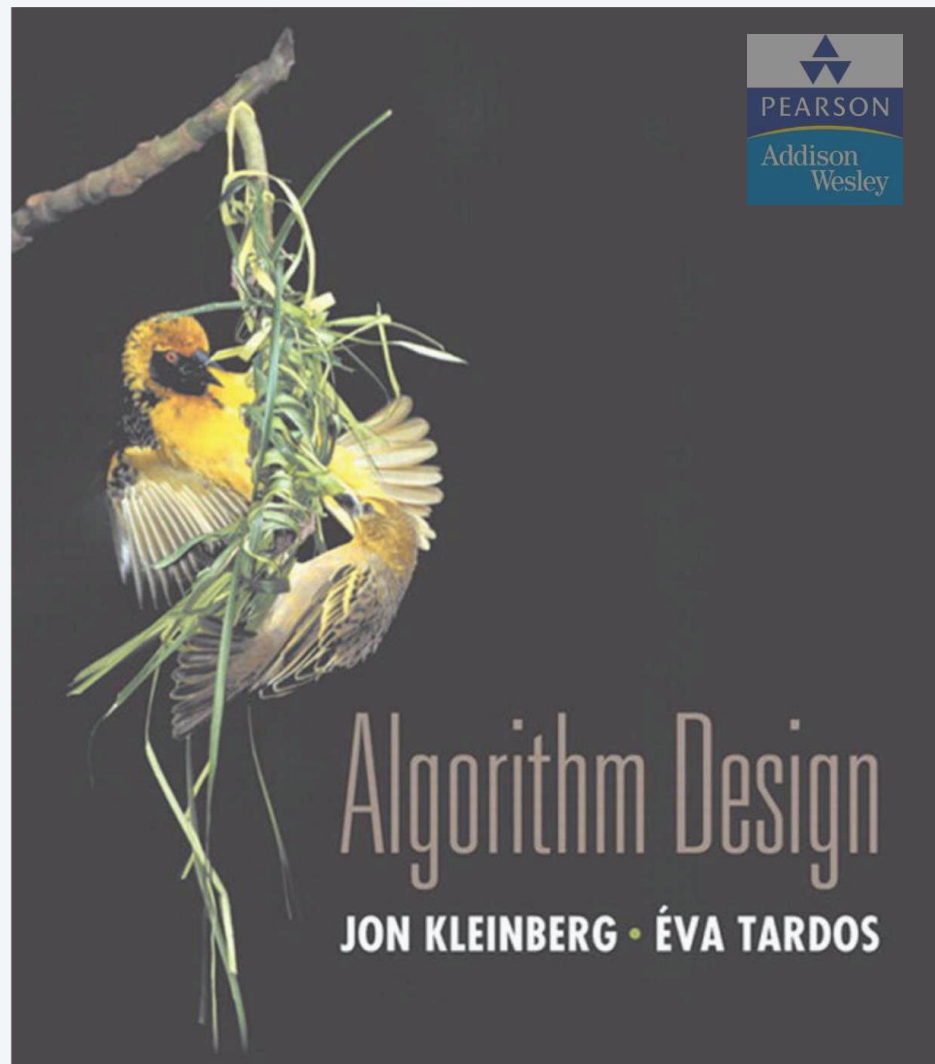
“no” instances of VERTEX-COVER
are solved correctly



vertex cover instance (k
= 2)



set cover instance
($k = 2$)



SECTION 8.2

8. INTRACTABILITY I

- *constraint satisfaction problems*

Satisfiability

Literal. A Boolean variable or its negation.

$$x_i \text{ or } \overline{x_i}$$

Clause. A disjunction of literals.

$$C_j = x_1 \vee \overline{x_2} \vee x_3$$

Conjunctive normal form (CNF). A propositional formula Φ that is a conjunction of clauses.

$$\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$$

SAT. Given a CNF formula Φ , does it have a satisfying truth assignment?

3-SAT. SAT where each clause contains exactly 3 literals

(and each literal corresponds to a different variable).

ESERCIZIO
DI
PROBLEMA
NP
COMPLETO

$$\Phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$$

yes instance: $x_1 = \text{true}$, $x_2 = \text{true}$, $x_3 = \text{false}$, $x_4 = \text{false}$

Key application. Electronic design automation (EDA).

Satisfiability is hard

Scientific hypothesis. There does not exist a poly-time algorithm for 3-SAT.

P vs. NP. This hypothesis is equivalent to **P $\neq$ NP** conjecture.

1. Perché risolvere 3-SAT in polinomiale implicherebbe risolvere tutti i problemi NP?

- 3-SAT è NP-completo.
- Definizione di NP-completo:
 - Il problema sta in NP (cioè le soluzioni si verificano in tempo polinomiale).
 - Ogni altro problema in NP si può ridurre in tempo polinomiale a 3-SAT.
- Questo significa: se hai un "oracolo" che risolve 3-SAT velocemente, puoi tradurre qualunque altro problema in NP in un'istanza di 3-SAT, risolverlo con l'oracolo, e ottenere la risposta.

✚ Quindi un algoritmo polinomiale per 3-SAT = algoritmo polinomiale per **tutti** i problemi NP.

E questo è esattamente il problema P vs NP.

2. Perché un insieme di clausole con 3 letterali è difficile?

- Non è difficile perché ci sono solo 3 variabili nella clausola: il punto è che l'intera formula ha **molte clausole e molte variabili**.
- Una clausola come $(x_1 \vee x_2 \vee x_3)$ è banalissima da verificare.
- La difficoltà nasce quando hai **tante clausole con variabili condivise**.
Devi trovare un'assegnazione unica per tutte le variabili che soddisfi **tutte le clausole contemporaneamente**.
- Questo crea un effetto "puzzle globale", in cui ogni decisione influenza molte clausole.

clausole.

3. Perché il brute force non è polinomiale?

- Con brute force devi testare tutte le possibili assegnazioni delle n variabili.
- Numero delle assegnazioni = 2^n .
- La complessità quindi è **esponenziale**, non polinomiale.

✚ Attenzione: non è che il brute force diventa polinomiale, anzi!

- Se $n = 10$, $2^n = 1024 \rightarrow$ gestibile.
- Se $n = 100$, $2^{100} \approx 10^{30} \rightarrow$ impossibile da calcolare in pratica.

Quindi il punto è:

- Polinomiale = $O(n^k)$, cresce "abbastanza lentamente" con n .
- Esponenziale = $O(2^n)$, cresce "troppo velocemente".

E per 3-SAT (e per tutti i problemi NP-completi noti) al momento abbiamo solo algoritmi esponenziali (o quasi-esponenziali), non polinomiali.

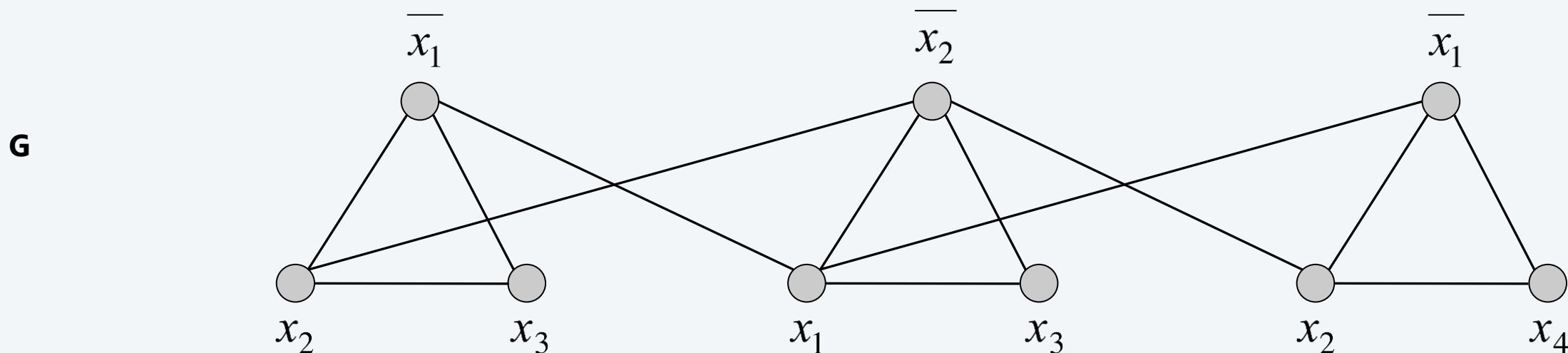
3-satisfiability reduces to independent set

Theorem. 3-SAT $\overset{\text{REDUCE}}{\leq_P}$ INDEPENDENT-SET.

Pf. Given an instance Φ of 3-SAT, we construct an instance (G, k) of INDEPENDENT-SET that has an independent set of size $k = |\Phi|$ iff Φ is satisfiable.

Construction.

- G contains 3 nodes for each clause, one for each literal.
- Connect 3 literals in a clause in a triangle.
- Connect literal to each of its negations.



k = 3

$$\Phi = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_4)$$

3-satisfiability reduces to independent set

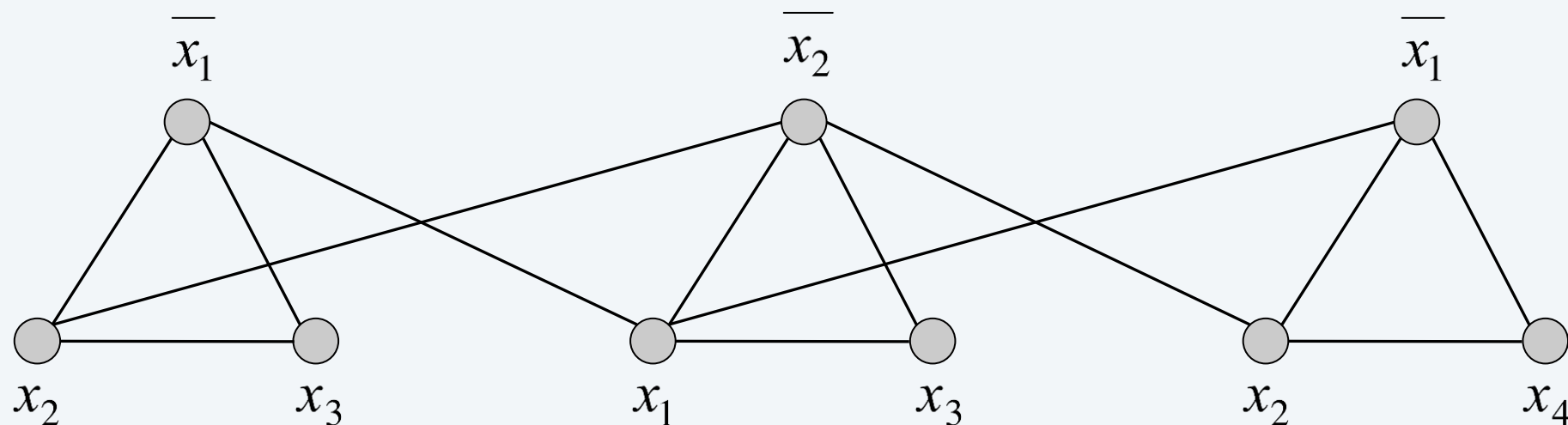
Lemma. Φ is satisfiable iff G contains an independent set of size $k = |\Phi|$.

Pf. $\Rightarrow$ Consider any satisfying assignment for Φ .

- Select one true literal from each clause/triangle.
- This is an independent set of size $k = |\Phi|$. ▪

“yes” instances of 3-SAT
are solved correctly

G



k = 3

$$\Phi = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_4)$$

3-satisfiability reduces to independent set

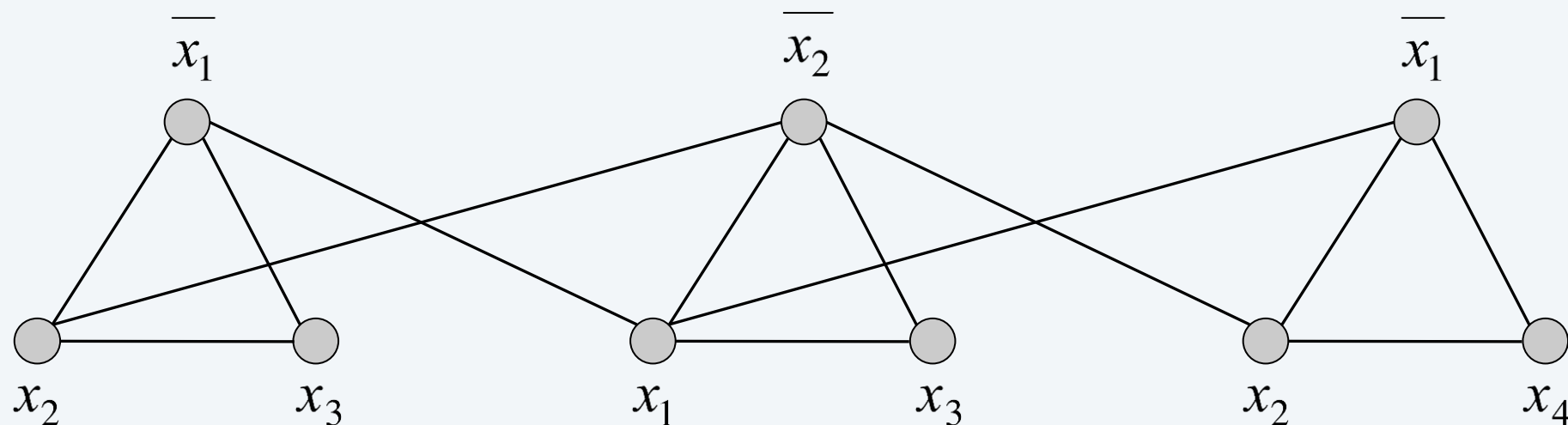
Lemma. Φ is satisfiable iff G contains an independent set of size $k = |\Phi|$.

Pf. $\Leftarrow$ Let S be independent set of size k .

- S must contain exactly one node in each triangle.
- Set these literals to *true* (and remaining literals consistently).
- All clauses in Φ are satisfied. ▪

“no” instances of 3-SAT
are solved correctly

G



k = 3

$$\Phi = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_4)$$

Review

Basic reduction strategies.

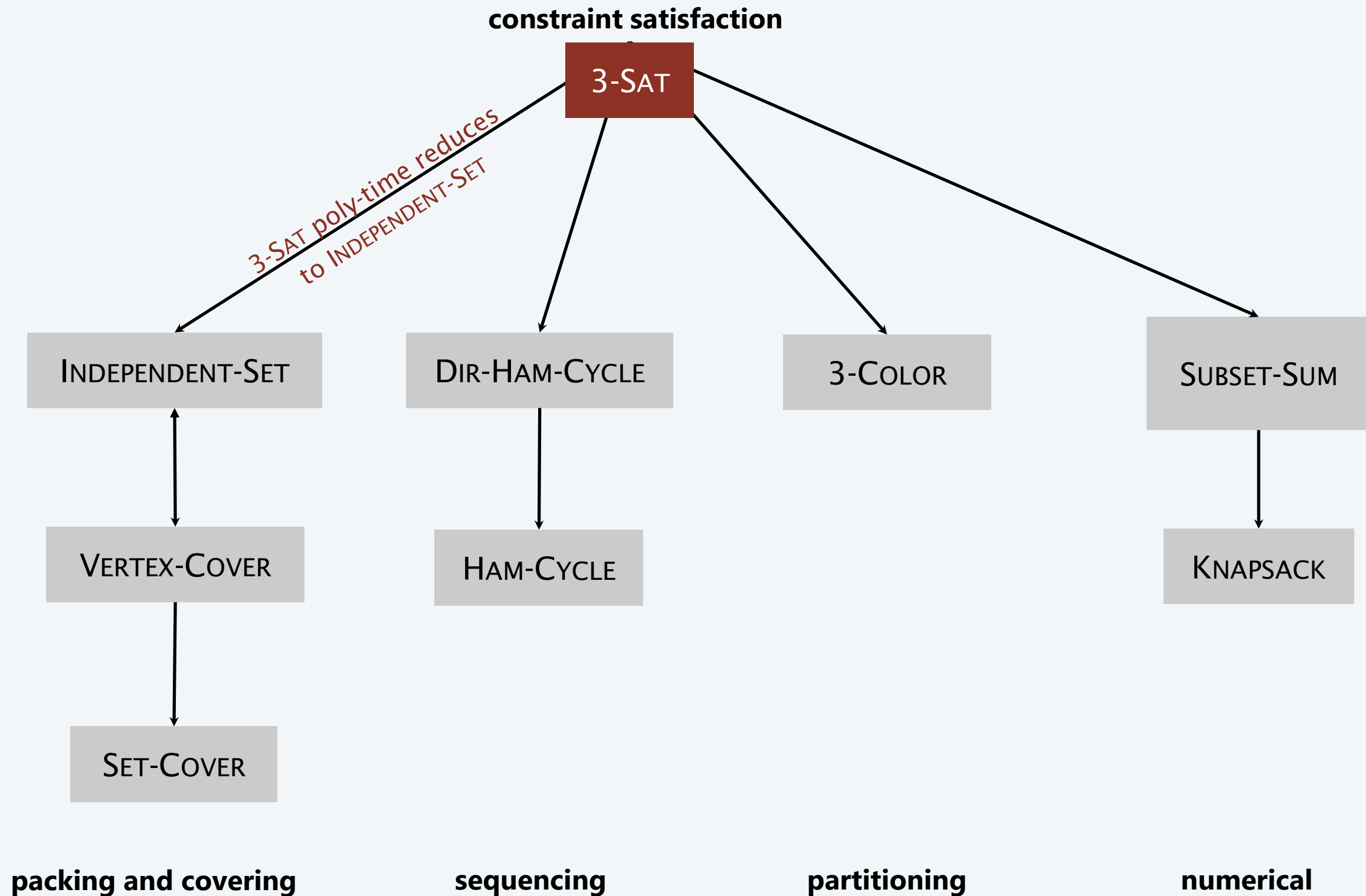
- Simple equivalence: $\text{INDEPENDENT-SET} \equiv_p \text{VERTEX-COVER}$.
- Special case to general case: $\text{VERTEX-COVER} \leq_p \text{SET-COVER}$.
- Encoding with gadgets: $3\text{-SAT} \leq_p \text{INDEPENDENT-SET}$.

Transitivity. If $X \leq_p Y$ and $Y \leq_p Z$, then $X \leq_p Z$.

Pf idea. Compose the two algorithms.

Ex. $3\text{-SAT} \leq_p \text{INDEPENDENT-SET} \leq_p \text{VERTEX-COVER} \leq_p \text{SET-COVER}$.

Poly-time reductions



DECISION, SEARCH, AND OPTIMIZATION PROBLEMS

Decision problem. Does there **exist** a vertex cover of size $\leq k$?

Search problem. **Find** a vertex cover of size $\leq k$.

Optimization problem. **Find** a vertex cover of **minimum** size.

Goal. Show that all three problems poly-time reduce to one another.

SEARCH PROBLEMS VS. DECISION PROBLEMS



VERTEX-COVER. Does there exist a vertex cover of size $\leq k$?

FIND-VERTEX-COVER. Find a vertex cover of size $\leq k$.

Theorem. $\text{VERTEX-COVER} \equiv_p \text{FIND-VERTEX-COVER}$.

Pf. $\leq_p$ Decision problem is a special case of search problem. ▀

Pf. $\geq_p$

To find a vertex cover of size $\leq k$:

- Determine if there exists a vertex cover of size $\leq k$.
- Find a vertex v such that $G - \{v\}$ has a vertex cover of size $\leq k - 1$.
(any vertex in any vertex cover of size $\leq k$ will have this property)
- Include v in the vertex cover.
- Recursively find a vertex cover of size $\leq k - 1$ in $G - \{v\}$. ▀

delete v and all incident edges

OPTIMIZATION PROBLEMS VS. SEARCH PROBLEMS

FIND-VERTEX-COVER. Find a vertex cover of size $\leq k$.

FIND-MIN-VERTEX-COVER. Find a vertex cover of minimum size.

Theorem. $\text{FIND-VERTEX-COVER} \equiv_p \text{FIND-MIN-VERTEX-COVER}$.

Pf. $\leq_p$ Search problem is a special case of optimization problem. ▀

Pf. $\geq_p$ To find vertex cover of minimum size:

- ▀ Binary search (or linear search) for size k^* of min vertex cover.
- ▀ Solve search problem for given k^* . ▀